UNIT-VII

THE EMBEDDED SYSTEM DEVELOPMENT

* *

* The Integrated Development Environment (IDE):

Integrated Development Environment (IDE) stands for an integrated environment for developing and debugging the target processor specific embedded firmware. IDE is a software package which bundles a 'Text Editor (Source Code Editor)', 'Cross-Compile C-for cross platform development and Compiler for same platform development)', 'Linker' and a 'Debugger'. IDE's provide interface to target board Emulators, Target processor's /controller Flash memory programmer etc,. IDE's can be either command line based or GUI based. Command line based IDEs may include little or less GUI support. The old version of TURBO C IDE for developing applications in C/C++ for x86 processor on Windows platform is an Example for generic IDE with Command Line Interface. GUI based IDEs provides a Visual Development Environment with mouse click support for each action. Such IDE's are generally known as Visual IDE's.

IDE's used in Embedded firmware development are slightly different from the generic IDE's used for high level language based development for desktop applications. MPLAB is an IDE tool supplied by microchip for developing embedded firmware using their PIC family of microcontrollers.

* Types of files generated on Cross-Compilation :

Cross-Compilation is the process of converting a Source Code written in high level language (like 'Embedded C') to target processor / controller understandable machine code.

* The Conversion of the code is done by software running on a processor / controller (e.g x86 processor based PC) which is different from the target processor. The software performing this operation is referred as the 'Cross-Compiler'.
* In a single word Cross-Compilation is the process of Cross platform Software / firmware development.
* Cross assembling is similar to Cross-Compiling; the only difference is that the code written in a target processor / controller specific assembly code is converted into its corresponding machine code. The application converting Assembly instruction to target processor / controller specific machine code is known as Cross-assembler.
* Cross-Compilation / Cross-assembling is carried out in different steps and the process generates various types of intermediate files.

The various files generated are :

List File (.LST file)

Listing file contains an abundance of information about the cross compilation process, like cross compiler details, formatted source text ('C' code), assembly code generated from the source file, symbol tables, errors and warnings detected during the Cross-Compilation process.

Preprocessor output file

This file contains the preprocessor output for the preprocessor instructions used in the source file preprocessor preprocessor output file is used for verifying the operation of macros and conditional preprocessor directives. It is a valid C source file. File extension of preprocessor output file is cross compiler dependent.

Object file (.OBJ file)

Cross-Compiling / assembling each source module (written in C/ assembly) converts the various Embedded C/Assembly instructio and other directives present in the module to an object (.OBJ) file. OMF51 or OMF2 are the two objects file formats supported by C51 Cross Compiler.

The list of some of the details stored in an object file is given bela

1. Reserved memory for global variables.
2. public symbol (variable and function) names.
3. External symbol references.
4. Library files with which to link.
5. Debugging information to help synchronize source lines with object code.

* Linker / Locater is responsible to assign an absolute memory location to the object code.

Map file (.MAP).

Linking and locating of re-locatable object files will also generate a list file called 'Linker list file' or 'map file'.

* It contains the information about the Link / Locate process and

is composed of a number of sections. The different sec... in a map file are cross compiler dependent.

HEX File (.HEX)

Hex file is the binary executable file created from the sour... code. The absolute object file created by the linker/locater is converted into processor understandable binary code. The utility used for converting an object file to a hex file is known as object to Hex file converter. Hex files embed the machine code in a particular format.

Intel HEX and Motorola HEX are the two commonly used hex file formats in Embedded applications. Intel HEX file is an ASCII text file. Intel HEX file is used for transferring the program and data to a ROM or EPROM which is used as code memory storage.

Motorola HEX file is also an ASCII text file where the HEX data is represented in ASCII format in lines.

* Disassembler / Decompiler :

Disassembler is a utility program which converts machine codes into target processor specific Assembly codes/ instructions. The process of converting machine codes into Assembly code is known as 'Disassembling'.

→ Decompiler is the utility program for translating machine codes into corresponding high level language instructions. Decompiler performs the reverse operation of Compiler / Cross-Compiler. The disassemblers/ decompilers for different family of processors/ Controllers are different. It is not possible for a disassembler/ decompiler to generate an Exact replica of the original assembly code.

SIMULATORS, EMULATORS AND DEBUGGING:

Simulators and Emulators are important tools used in embedded system development. Both terms sound alike and are little confusing.

* Simulator is a software tool used for simulating the various conditions for checking the functionality of the application firmware.
* The Integrated Environment (IDE) itself will be providing simulator support and they help in debugging the firmware for checking its required functionality.

Simulators:

Simulators simulate the target hardware and the firmware execution can be inspected using simulators. The features of simulator based debugging are listed below.

(1) purely software based.
(2) Doesn't require a real target system.
(3) very primitive (lack of featured I/O support. Everything is simulated one).
(4) Lack of realtime behaviour.

Advantages of simulator based debugging:-

Simulator based debugging techniques are simple and straight-forward. The major advantages of simulator based firmware debugging techniques are explained below.

No need for Original Target Board:-

Simulator based debugging technique is purely software oriented. IDE's software support simulates the CPU of the target board. User only needs to know about the memory map of

various devices within the target board and the firmware should be written on the basis of it. Since the real hardware is not required, firmware development can start well in advance immediately after the device interface and the memory maps are finalised. This saves development time.

Simulate I/O peripherals :-

Simulator provides the option to simulate various I/O peripherals. Using simulator's I/O support you can edit the values for I/O registers and can be used as the input/output value in the firmware execution. Hence it eliminates the need for connecting I/O devices for debugging the firmware.

Simulates Abnormal conditions :-

With simulator's simulation support you can input any desired value for any parameter during debugging the firmware and can observe the control flow of firmware. It really helps the developer in simulating abnormal operational environment for firmware and helps the firmware developer to study the behaviour of the firmware under abnormal input conditions.

Limitations of simulator based debugging :

Deviation from Real behaviour :- Simulation-based firmware debugging is always carried out in a development environment where the developer may not be able to debug the firmware under all possible combinations of input. Under certain operating conditions we may get some particular result and it need not be the same when the firmware runs in a production environment.

Lack of real timeliness :- The major limitation of simulator based debugging is that it is not real-time in behaviour. The debugging is developer driven and it is no way capable of creating a real time behaviour. Moreover in a real application the I/O condition may be varying (or) unpredictable.

Target Hardware Debugging:

→ Even though the firmware is bug free and everything is intact in the board, your embedded product need not function as per the expected behaviour in the first attempt for various Hlw related reasons like dry soldering of components, missing connections in the PCB due to any un-noticed errors in the PCB layout design, misplaced components, signal corruption due to noise etc. The only way to sort out these issues and figure out the real problem creator is debugging the target-board.

→ H/w debugging is not similar to firmware debugging. H/w - debugging involves the monitoring of various signals of the target board (address / data lines, port pins, etc), checking the interconnection among various components, circuit continuity checking etc. The various Hlw debugging tools used in Embedded Product Development are explained below.

1) Magnifying Glass (Lens):

→ As we noticed that watch repairer wearing a small magnifying glass while engaged in repairing a watch. They use the magnifying glass to view the minute components inside the watch in an enlarged manner so that they can easily work with them.

→ Hy, magnifying glass is the primary Hlw debugging tool for an embedded Hlw debugging professional.

→ A magnifying glass is a powerful visual inspection tool. with a lens, the surface of the target board can be examined thoroughly for dry soldering of components, missing components, improper placement of components, improper soldering, track (PCB connection) damage, short of tracks etc.

→ The magnifying station incorporates magnifying glasses attache to a stand with CFL tubes for providing proper illumination fo

inspection. The station usually incorporates multiple magnifying lens. The main lens acts as a vitual inspection tool for the entire Hlw board whereas the other small within the station is used for magnifying a relatively area of the board which requires thorough inspection.

2) Multimeter:

→ A multimeter is used for measuring various electrical quantities like voltage (Both AC & DC), current (DC & AC), 'R', 'C', continuity checking, transistor checking, cathode and anode identification of diode etc.

→ Any multimeter will work over a specific range for each measurement. A multimeter is the most valuable tool in the toolkit of an embedded Hlw developer.

→ It is mainly used for checking the circuit continuity b/w diff. points on the board, measuring the supply voltage, checking the signal value, polarity etc.

→ Both analog and digital versions of a multimeter are availabl The digital version is preferred over analog due to its readability, accuracy etc.

→ Fluke, Rishab, Philips etc are the manufacturers of commonly available high quality digital multimeters.

3) Digital CRO:

→ cathode Ray Oscilloscope (CRO) is used for waveform capturin and analysis, measurement of signal strength etc. By connecting the point under observation on the target board to the channels of the oscilloscope, the waveforms can be captured and analysed for expected behaviour.

→ CRO is a very good tool in analysing interference noise in the Power supply line and other signal lines.

→ Monitoring the crystal oscillator signal from the target board is a typical example of the usage of CRO for waveform capturing and analysis in target board debugging.

's are available in both analog and digital versions. Though digital CROs are costly, feature wise they are best suited for target board debugging applications.

→ Digital CRO's are available for high freq. support and they also incorporate modern techniques for recording waveform over a period of time, capturing waves on the basis of a configurable event (triggers) from the target board.

→ Most of the modern digital CRO's contain morethan one channel & it is easy to capture and analyse various signals from the target board using multiple channels simultaneously. Various measurements like phase, amplitude etc is also possible with CRO's

→ Tektronix, Agilent, philips etc are the manufacturers of high precision good quality digital CRO's.

4) Logic Analyser:

→ It is used for capturing digital data (logic 1 and 0) from a digital circuitry whereas CRO is employed in capturing all kinds of waves including logic signals.

→ Another major limitation of CRO is that the total no. of logic-signals / waveforms that can be captured with a CRO is limited to the no. of channels.

→ A logic analyser contains special connectors & clips which can be attached to the target board for capturing digital data.

→ A logic analyser captures the states of various port pins, address bus and data bus of the target processor / controller etc & it give an exact reflection of what happens when a particular line of firmware is running. This is achieved by capturing the address line logic and data line logic of target H/w.

→ It contain provisions for storing captured data, selecting a desired region of the captured waveform, zooming selected region of the captured waveform etc.

- there are number of codes to be linked

→ Tektronix, Agilent etc are the giants in the logic analysers market.

5) Function Generator:

→ Function generator is not a debugging tool. It ~~is~~ an i/p signal simulator tool. It is capable of producing various periodic waveforms like sine wave, square wave, saw-tooth wave etc. with diff. frequencies and amplitude.

→ Sometimes the target board requires some kind of periodic waveform with a particular freq. as i/p to some part of the board. Thus, in a debugging environment, it serves the purpose of generating and supplying required signals.

* Boundary Scan:

→ As the complexity of the H/w ↑, the no. of chips present in the board and the interconnection among them may also ↑.

→ The device packages used in the PCB become miniature to reduce the total board space occupied by them and multiple layers may be required to route the interconnections among the chips.

→ with this, for the PCB it will be very difficult to debug the H/w using magnifying glass, multimeter etc. to check the interconnection among the various chips.

→ Boundary scan is a technique used for testing the interconnection among the various chips, which support JTAG interface, present in the board. chips which support boundary scan associate a boundary scan cell with each pin of the device.

→ A JTAG port which contains the 5 signal lines namely TDI, TDO, TCK, TRST and TMS form the Test Access port (TAP). ~~for~~

TDI = Test Data In – used for sending debug commands serially from remote debugger to the target processor

TDO = Test Data Out – Transmit debug response to remote debugger from target CPU

TCK = Test clock – synchronises the serial data transfer

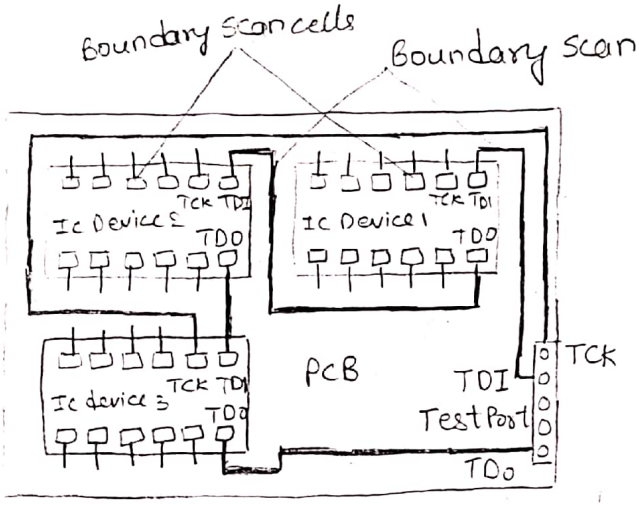


fig:- JTAG based boundary scanning for H/w testing

TMS = Test mode select - sets the mode of testing.
TRST = Test reset - It is an optional signal line used for resting the target CPU.

→ Each device will have its own TAP. The PCB also contains a TAP for connecting the JTAG signal lines to the ext. world.

→ A boundary scan path is formed inside the board by interconnecting the devices through JTAG signal lines.

→ The TDI pin of the TAP is connected to the TDI pin of first device. The TDO pin of the first device is connected to the TDI pin of the second device. In this way all devices are interconnected and the TDO pin of the last JTAG device is connected to the TDO pin of the TAP of the PCB.

→ The clock line TCK & the Test mode select (TMS) line of the devices are connected to the clock line & test mode select line of the test access port of the PCB.

→ Boundary scancell:- It is a multipurpose memory cell. It is associated with the i/p Pins of an IC is known as 'I/P cells' & the boundary scan cell associated with the o/p Pins of an IC - 'o/p cells'. It can be used for capturing the i/p pin signal state and passing it to the internal circuitry, capturing the signals from the internal circuitry and passing it to the o/p Pin, & shifting the data Rxed from the TDI pin of the TAP. It associated with

the pins are interconnected & they form a chain from the TDI Pin of the device to its TDO pin. These can be operated in normal, capture, update & shift modes.

→ In the normal mode - the i/p of the boundary scan cells appears directly as its o/p.

→ In the capture mode - the it associated with each i/p pin of the chip captures the signal from the respective pins to the cell & it associated with each o/p pin of the chip catures the signal from the internal circuitary.

→ In the update mode - it associated with each i/p pin of the chip Passes the already captured data to the internal circuitry & it associated with each o/p pin of the chip passes the already captured data to the respective o/p pin.

→ In the shift mode - data is shifted from TDI pin to TDO pin of the device through the boundary scan cells.

→ Instruction Register, Bypass register, identification register etc are examples of boundary scan related registers for facilitating the boundary scan operation.

→ The instruction register is used for holding & processing the instruction Rxed over the TAP.

→ The bypass register is used for bypassing the boundary scan Path of the device and directly interconnecting the TDI Pin of the device to its TDO. It disconnects a device from the boundary scan path

→ Diff. instructions are used for testing the interconnections and the functioning of the chip.

→ Extest, Bypass, sample and preload, Intest etc are examples for instructions for diff types of boundary scan tests, where as the instruction Runbist is used for performing a self test on the internal functioning of the chip. The Runbist instruction produce a pass/fail result.

Boundary Scan Description Language (BSDL):

→ BSDL is a language similar to VHDL, which describes the boundary scan implementation of a device & is used for implementing boundary scan tests using JTAG. BSDL file is a file containing the boundary scan description for a device in boundary scan description language, which is supplied as i/p to a boundary scan tool for generating the boundary scan test cases.

Development Process and Hardware-Software

The following figure shows the development process of an Embedded system and fig(b) shows edit-test-debug cycle during the implementation phase of the development process. Whereas the processor part once chosen remains fixed, the application software codes have to be perfected by a number of runs and tests. The cost of the processor is quite small, the cost of developing final targeted system is quite high and larger time frame.

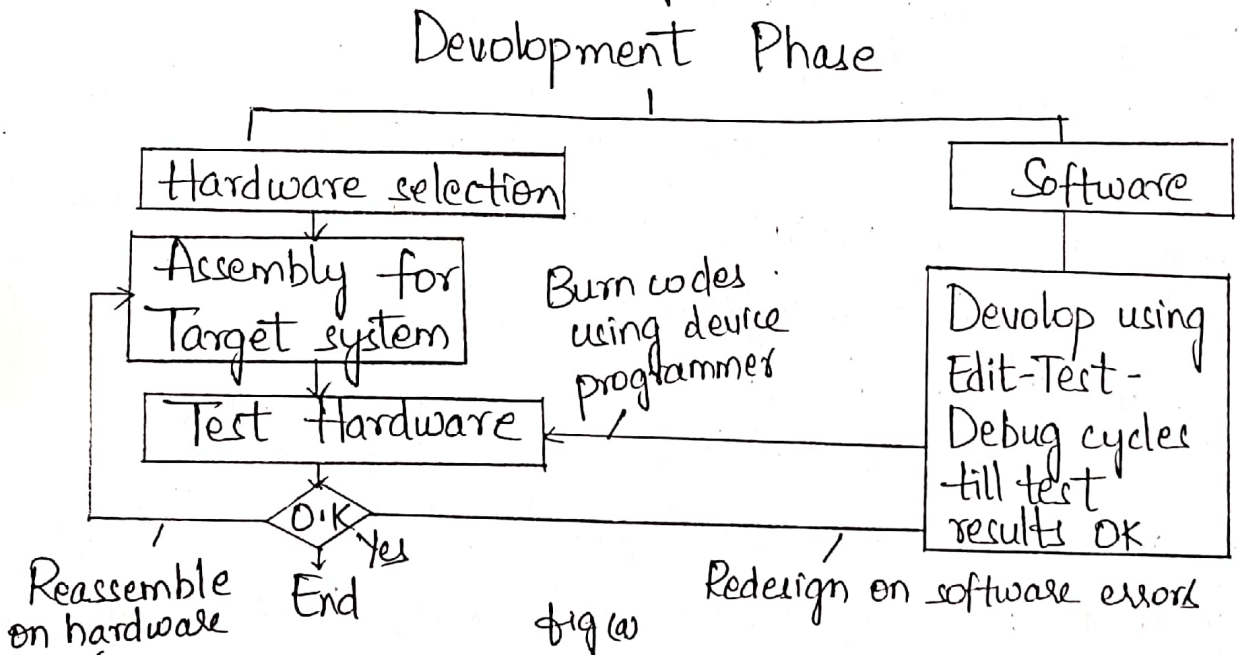
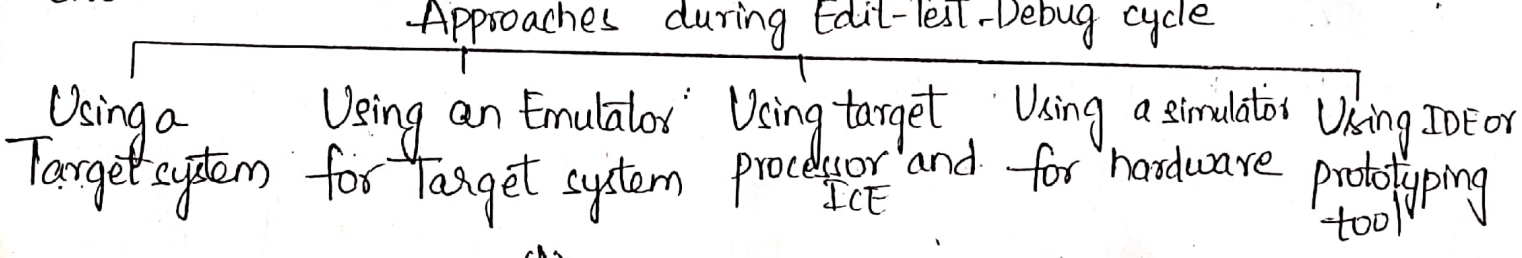


fig (a)



(b)

The developer uses four main approaches to the edit test debug cycles

i) An IDE or prototype tool
ii) A simulator without any hardware

iii. Processor only at the target system and uses an in-between ICE (In Circuit Emulator)
iv. Target system at the last stage

Software Tools

The tools are required for the application software high level language programming. Also required are RTOS, testing debugging, Assembly language programming and system integration tools.

The following are the software tools in software and hardware implementation for embedded system

Development Kit, Source code Engineering software, RTOS IDE, prototyper, Compiler, Assembler, Cross assembler, Cross compiler, locator, editor, interpretor

Source Code Engineering Tool: It is of great help for source code development, compiling and cross compiling. The tools are commercially available for embedded c/c++ code engineering, testing and debugging.

The features of a typical tool are comprehension, navigation and browsing, editing, debugging, configuring (disabling and enabling the c++ features) and compiling

A tool for C and c++ is SNiFF+. It is from WindRiver systems. The main features of the tool are

1) It provides easy and automated search and development
2) It automatically removes error-prone and unused tasks.
3. It browses through object component relationship.
4. It finds the complete effect of any code change on the source code.

Integrated Development Environment:

IDE consists of simulators with editors, compilers, assemble etc emulators with logic analysers and EPROM/EEPROM application codes burner. It has the following features

1) It has a facility for defining a processor family as well as defining its version.
2. It has the facility of a user definable assembler to support a new version (or) type of processor
3. It provides multiuser environment
4. It debugs by single stepping. It has the facility for synchronising the internal peripherals.
5. It verifies the performance of target system.

HOST AND TARGET MACHINES

During the development process, a host system is used before locating and burning the codes in the target board. The target board hardware and software is later copied to get the final embedded system, which will function exactly as the one tested and debugged and finalized during the development process.

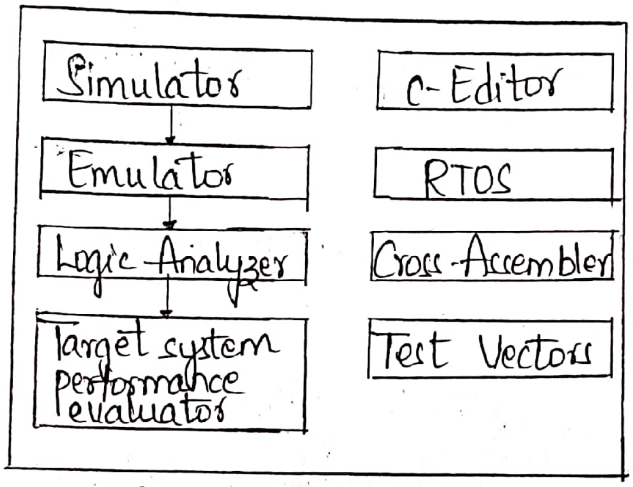
Using a Host System: Host system is a PC or workstation or laptop. It has the following hardwares

(1) High performance processor with caches
(2) Large RAM memory
(3) ROMBIOS [Read Only Memory Basic Input Output System]
(4) Very large memory on disk
(5) Keyboard.

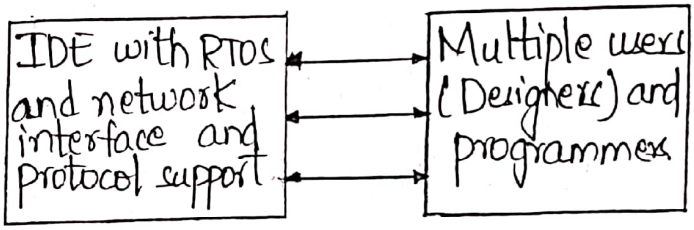
(6) Display monitor
(7) Mice
(8) Network Connection

It is a full fledged computer. It has software tools and must include the following

(1) Program development Kit for a high-level language program or IDE
(2) Host processor compiler and Cross compiler
(3) Cross-Assembler.



(a) Simple IDE



(b) Sophisticated IDE

Program development Tool Kit

It has an editor which is used for writing 'C' codes or assembly mnemonics or C++ or Java using the Keyboard of the host system for entering the program. Using GUI it allows the entry, addition, deletion, inserting etc. It creates a source file which saves the edited file.

An Interpretor does expression by expression translation to machine executable codes.

A Compiler converts high level language to machine executable codes. These codes are called Object codes. e.g.

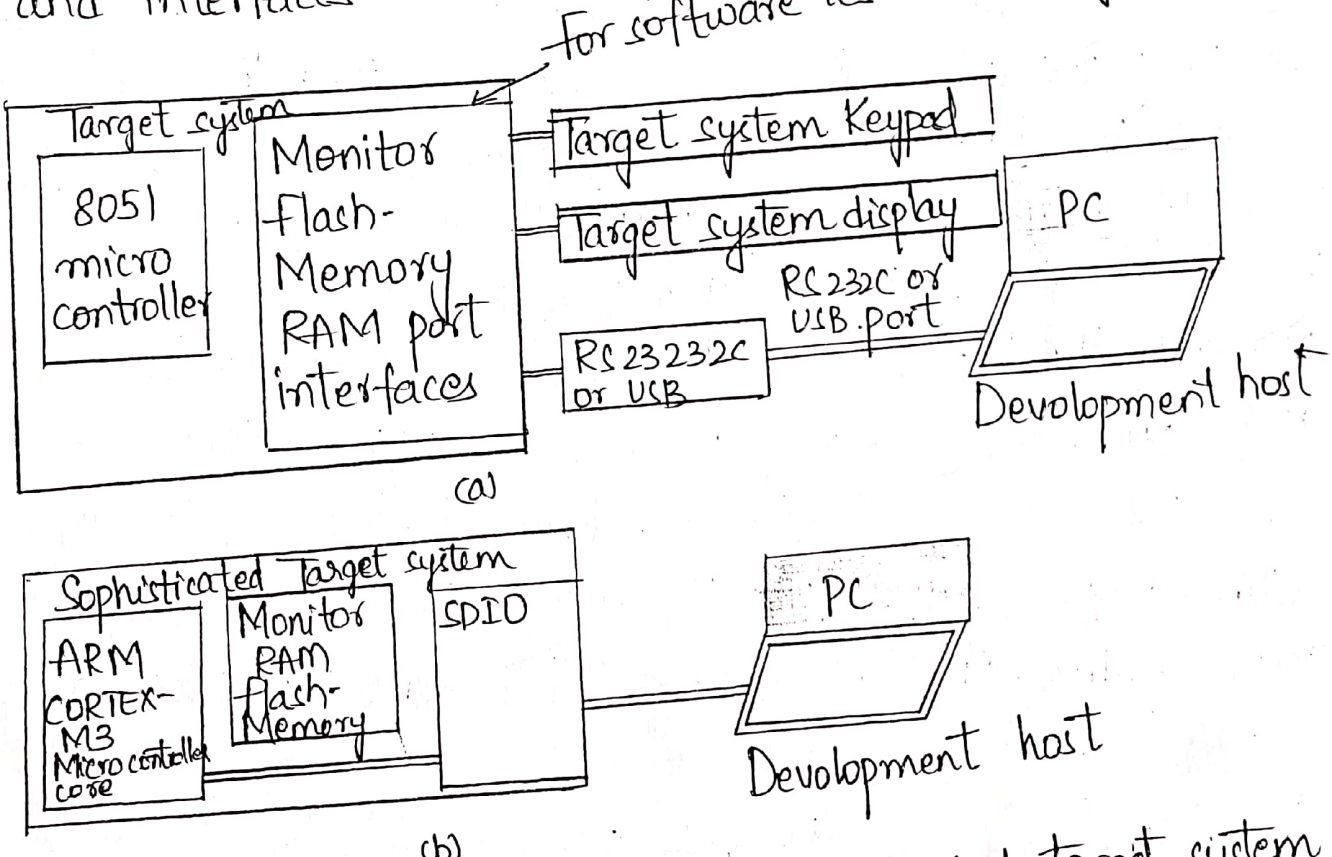
→ A dissembler translates the object codes into mnemonic form of assembly language

→ An Assembler is a program that translates assembly language to machine codes.

Cross-Compiler: A Cross compiler is a compiler that creates binary executable files for target system processor.

Cross Assembler: It converts object codes(or) executable codes for a processor to other codes of another processor and vice versa.

Target System: A Target system has a processor, ROM memory for ROM image of embedded software, RAM for stack, temporary variables and memory buffers, peripherals and interfaces.



fig(a) Simple Target system fig(b) Sophisticated target system

Linking and Locating Software:

A linker links the compiled codes of application softw object codes from library and os Kernal. Linking is necessary because there are number of codes to be lin

for the binary file. for example there are the standard codes to program a delay task for which there is a reference in the assembly language program. The codes for the delay must link with the assembled codes. The delay code is sequential from certain beginning address. The assembly software code is also sequential from a certain beginning address. Both the codes are present at the distinct and the available addresses in the system. A linker links these The linked file in binary for run on a computer is commonly known as executable file or simply '.exe' file. After linking there has to be reallocation of the sequences of placing the codes before the actual placements of codes in memory

A program is loaded in a computer RAM. The loader progra performs the task of reallocating the codes after finding the physical memory address available at a given instant. The loader is a part of OS and places codes into memory after reading the ".exe" file. This step is necessary because the available memory addresses may not start from 0x000 and the binary codes have to be loaded at the different addresses during the run. The loader finds the appropriate start address. After loading into RAM the program is ready to run.

When the code embeds into ROM or flash the system design process locates thes codes as ROM image. The codes are placed permanently at the actually available addresses in flash ROM

TARGET HARDWARE DEBUGGING:-

Even though the firmware is bug-free and everything is intact in the board your embedded product need not function as per the expected behaviour in first attemp for various hardware related problems like soldering of component, misplaced components, signal corruption due to noise etc. the only way to sort out this problem is debugging the target board

*the various hardware debugging tools used in Embedded product development are

(1) Magnifying glass (lens)
(2) Multimeter
(3) Digital CRO
(4) Logic Analyser
(5) function Generator

(1)- Similar to a watch repairer, magnifying is the primary hardware debugging tool for embedded hardware debugging professionals. A lens is a poor visual inspection tool with which the surface of target board is examined thoroughly for dry soldering of components, missing components, improper placement of components etc.

Now a days high quality magnifying stations are available for visual inspection. These stations contains lens attached to a stand with CFL tube for proper inspection. the main lens acts as a visual inspection tool for entire H/w board & small lens are relatively used for small areas of board for inspection.

(2) - A multimeter is used for measuring various electrical measurements like volts(ac & dc), current(ac & dc), resistance, capacitance, transistor checking etc. Any multimeter will work over a specific range of measurement. It is the most valuable tool in the toolkit of an embedded H/w developer. It is primary debugging tool & almost all developers start debugging the H/w using this. It is mainly used for checking signal value, measuring supply voltage, polarity etc.
Both analog and digital versions are available but digital version is preferred over because of readability, accuracy etc.
Manufacturers - fluke, philips etc.

(3) -
*more sophisticated than multimeter.
*It is used to capture & analyse the measurement of signal strength etc.
*By connecting observation point on target board to channels of CRO, the waveform can be captured

& analysed for expected behaviour.

*It is a very good tool in analysing interference noise in power supply line and other signal lines.

*Monitor crystal signal is one typical e.g for this

*CRO's are available in both analog & digital versions. both digital is costly but preferable for applications

* Digital CRO's are available for high frequency support and also they incorporate modern techniques.

*Digital CRO's contains more than one channel so that it is easy to capture & analyse signals.

* various measurements like phase, amp etc are possible with CROs.

Manufacturers - Agilent, philips etc.

(4)-

*It is big brother of digital CRO.

* logic analyser is used to capacuting digital data (0 & 1) from digital circuitry whereas CRO is for capturing all kinds of waves including logic signals

* Another major limitation of CRO is total no. of logic signals are limited to no. of channels

* It contains special connectors and clips which are attached to the target board for capturing digital data.

* logic analysers give an exact reflection of what happens when a particular line of firmware is running.

*It is achieved by capturing the address line logic & data line logic of target h/w.

* Most modern logic analyzers contain provisions for storing captured data, selecting a desired region of captured w/f, zooming selected region of captured w/f, etc.

Agilent - manufacturers.

(5)-

*Not a debugging tool, it is an i/p signal simulator tool.

* It is capable of producing various periodic waveform like sine, square, saw tooth etc with diff freq & amp.

*Sometimes target board may require some kind of periodic w/f with a particular freq as i/p to some part of board.

*thus function generator serves the purpose of generating and supplying required signals.

BOUNDARY SCAN:-

As the complexity of H/w increases, the no. of chips present in board & interconnection among them also increases the device package used in PCB become miniature to reduce total board space occupied by them and multiple layers may be required to route interconnection among the chips.

Boundary scan is a technique used for testing the interconnection among the various chips, which support JTAG interface present in board chips which support boundary scan associate a boundary scan cell with each pin of the device.

The JTAG port contains 5 signal lines namely TDI, TDO, TCK, TRST & TMS form the Test access port (TAP) for a JTAG suported chip. Each device will have its own TAP.

A boundary scan path is formed inside the board by interconnecting the device through JTAG signal lines. The TDI pin of TAP of PCB is connected to TDI pin of first device. The TDO pin of first device are connected to TDI pin of second device. In this way all devices are interconnected and TDO pin of last JTAG device is connected to TDO pin of TAP of PCB. the clock line TCK and Test Mode select (TMS) line of devices are connected to clock line & Test mode select line of Test Access port of PCB. They form a boundary

scan path.

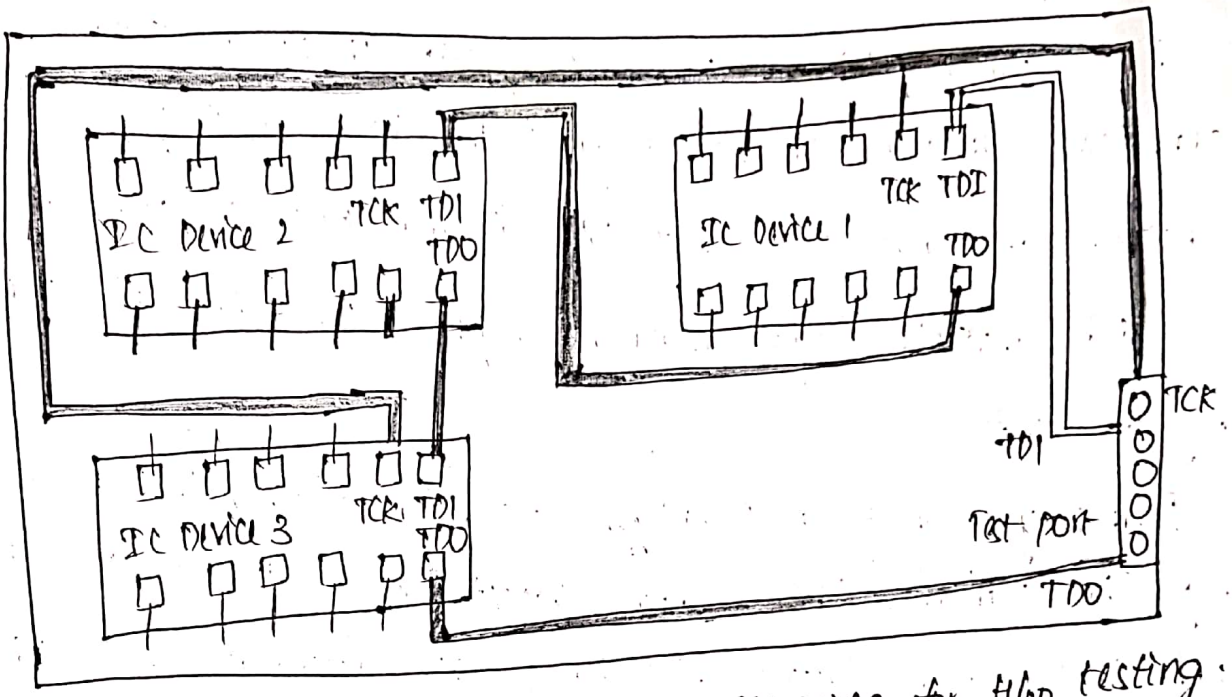


fig: JTAG based boundary scanning for H/w testing.

*Each pin of the device associates a boundary scan cell with it. the boundary scan cell is multipurpose memory cell. the boundary scan cell associated with input pins of an IC known as 'Input cells', the boundary cells associated with output pin is 'output cells'.

the boundary scan cells can be operated in Normal, capture, update & shift modes.

Normal mode - In this the i/p of the boundary scan cell appears directly at its output.

Capture mode - the boundary scan cell associated with each input pin of the chip captures the signal from the respective pins to the cell and boundary scan cell associated with each o/p pin of chip captures the signal from internal circuitry.

Update mode - the boundary scan cell associated with input pin of the chip passes the already captured data to the internal circuitry and the boundary scan cell associated with each output pin of the chip passes the already captured data to the respective o/p pin.

Shift mode - In this the data is shifted from TDI pin to TDO pin of device through the boundary scan cells.

IC supporting boundary scan contain additional boundary scan related registers for facilitating the boundary scan operation. Instruction Register, Bypass register, Identification register etc.

* Instruction Register is used for holding and processing the instruction received over the TAP.
* the bypass register is used for bypassing the boundary scan path of the device.
* Extest, Bypass, sample and preload etc are examples for instructions for different types of boundary scan tests.

Boundary scan description language (BSDL) is used for implementing boundary scan tests using JTAG. BSDL is the subset of VHDL.

UNIT-6.

Utility Tools : writing code

(1.) The main software in a editor or integrated development environment (IDE) :-

source code is typically written with a tool such as a standard ASCII text editor or an IDE located on the host development platform, as shown in fig. An IDE is a collection of tools, including an ASCII text Editor, integrated into one application user interface. while any ASCII text Editor can be used to write any type of code, independent of language and platform, an IDE is specific to the platform and is typically provided by the IDE's Vendor, a hardware manufacturer (in a starter kit that bundles the hardware board with tools such as an IDE or text editor), OS Vendor, or language Vendor (java, C, etc.)

The final phases of embedded Design:
Implementation and Testing

- **Implementing the Design :-**

The key development tools in embedded design can be located on the host or on the target, or can exist standalone. These tools typically fall under one of three categories : utility, Translation and debugging tools.

→ utility tools are general tools that aid in software or hardware development, such as editors, vcs that manages software files, and Rom burners, that allows software to be put onto Roms. Translation tools convert code a developer intends for the target into a form the target can execute, and debugging tools can be used to track down and correct bugs in the system.

(2.) Computer-Aided Design (CAD) and the Hardware

CAD tools are commonly used by hardware engineers to simulate circuits at the electrical level in order to study a circuit's behavior under various conditions before they actually build the circuit.

Because of the importance of and costs associated with designing hardware, there are many industry techniques in which CAD tools are utilized to simulate a circuit. Give a complex set of circuits in a processor or on a board, it is very difficult, if not impossible to perform a simulation on the whole design so a hierarchy of simulators and models is typically used. In fact, the use of models is one of the most critical factors in hardware design, regardless of the efficiency or accuracy of the simulator.

At the highest level, a behavioral model of the entire circuit. This behavioral model can be created with a cad tool that offers this feature (or) can be written in a standard programming language.

(3) Translation Tools - Preprocessors, INterpreters Compilers, Linkers

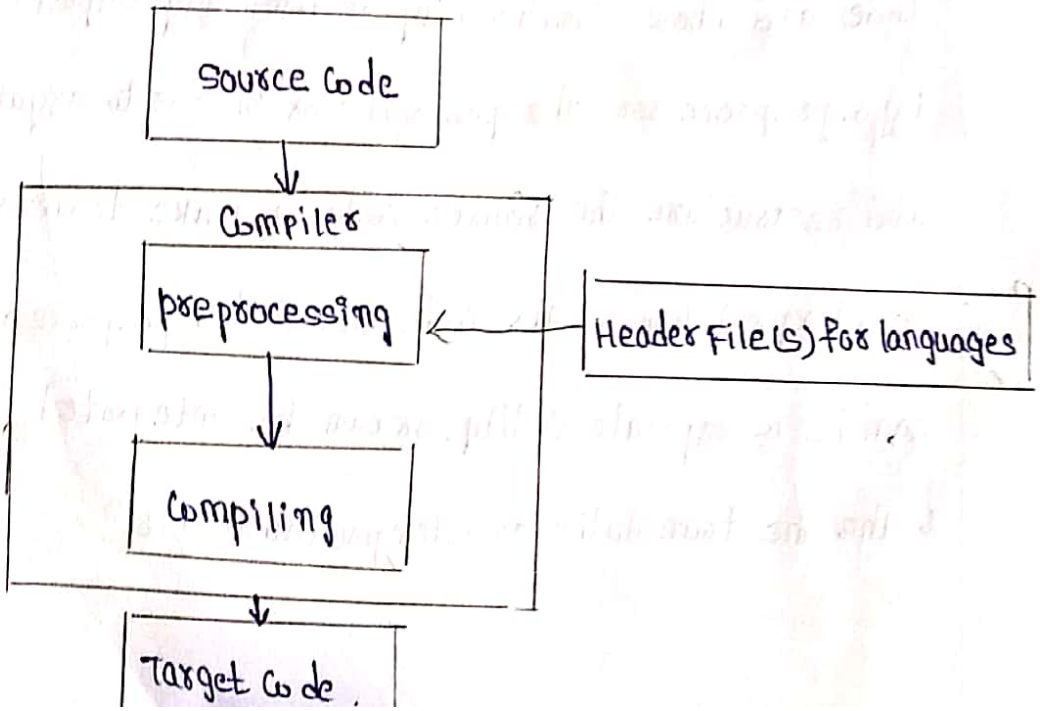
After the source code has been written, it needs to be translated into machine code since machine code is the only language the hardware can directly execute All other languages need development tools that generate the corresponding machine code the hardware will understand This mechanism usually includes one or some combination of preprocessing, translation, and/or interpretation machine code generation techniques. these mechanisms are implemented within a wide variety of translating development tools

* PREprocessing is an optional step that occurs before either the translation or interpretation of source code and whose functionality is commonly implemented by a preprocessor. the preprocessor role is to organise and restructure the source code to make translation or interpretation of this code easier. the preprocessor can be a separate entity, or can be integrated within the translation or interpretation unit

(ix) many languages convert source code, either directly or after having been preprocessed to target code through the use of a compiler a program which generates some target language (machine code Java byte code, etc) from the source language (assembly, C, Java, etc)

(x) A compiler typically translates all of the source code to a target code at one time. As is usually the case in embedded systems, most compilers are located on the programmer's host machine. and generate target code for hardware platforms that differ from the platform of compiler the compiler actually running on. these compilers are commonly referred to as cross compilers In the case of assembly, an assembly compiler is a specialized cross-compiler referred to as an assembler and will always generate machine code. other high level language

Compilation diagram:>

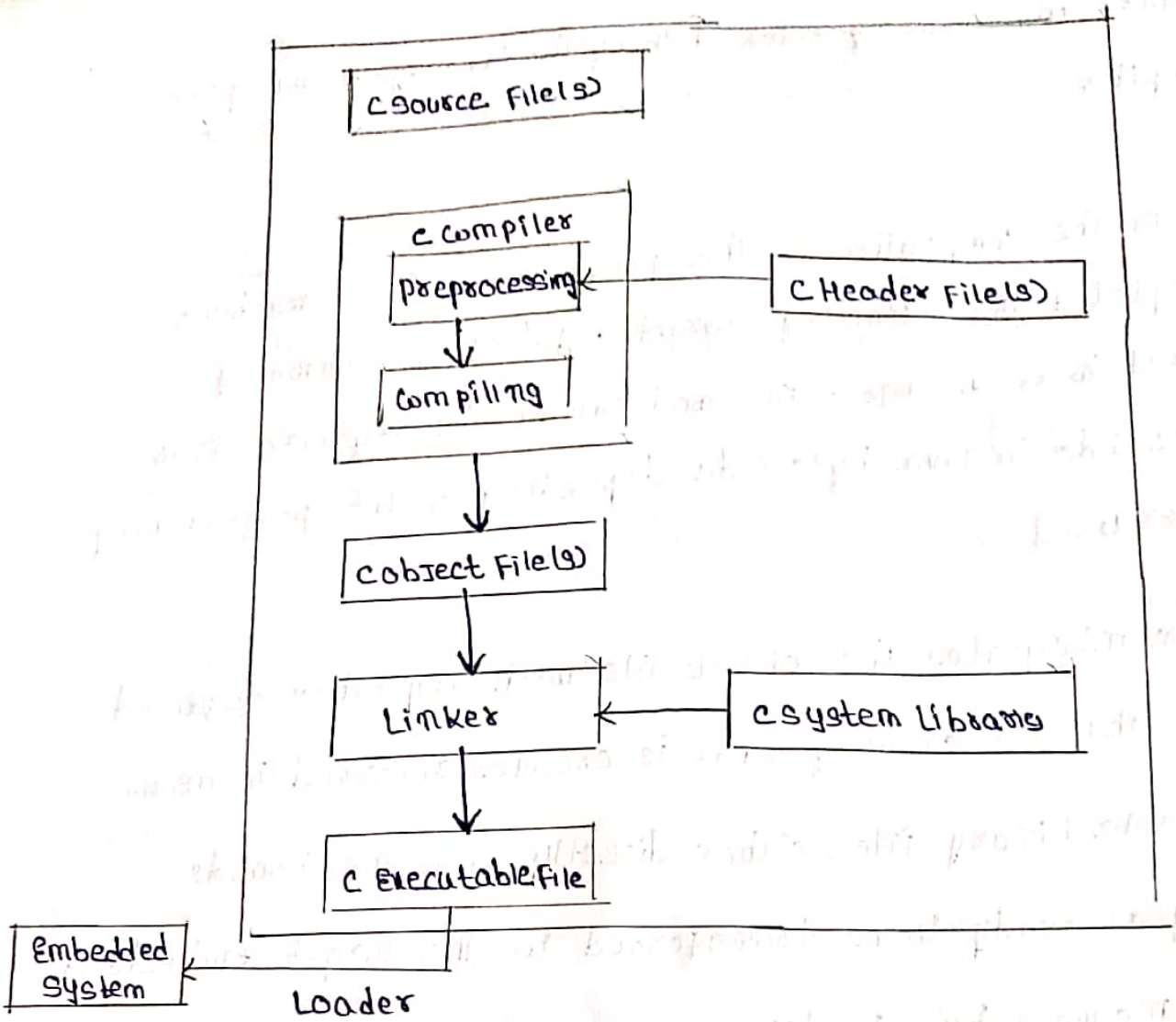


Compilers are commonly referred to by the language name plus "Compiler"

* After all the compilation on the programmer's host machine is completed the remaining target code file is commonly referred to as an object file and can obtain anything from machine code to Java byte code depending on the programming languages used

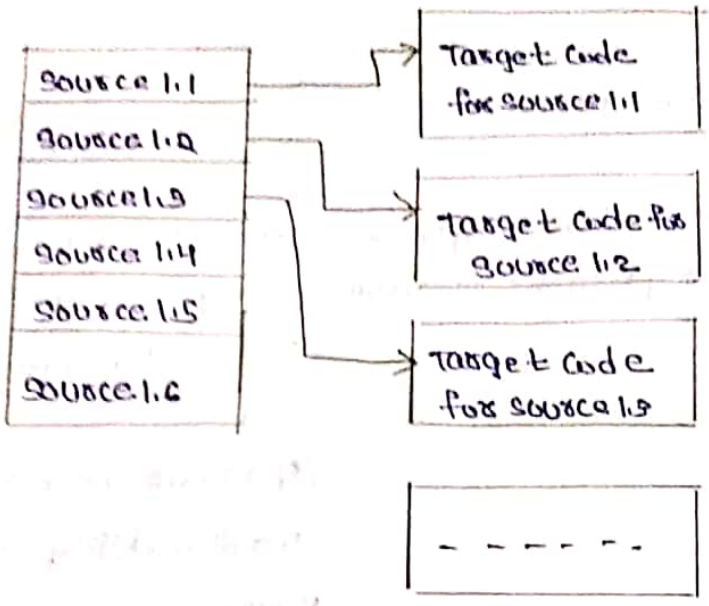
* A linker integrates the object file with any other required system libraries creating what is creates referred to as an Executable binary file, either directly onto the boards memory or ready to be transferred to the target embedded systems memory by a loader

* One of the fundamental strengths of a translation process is based upon the concept of software placement also referred to as object placement the ability to divide software into modules and relocate these modules of code and data anywhere in memory



C Example compilation | linking steps and object file result

* An interpreter generates machine code one source code line at a time from source code or target code generated by an intermediate compiler on the host system



Interpretation diagram

(4.) Debugging tools :-

Debugging code is probably the most difficult task of the Development cycle. Debugging is primarily the task of locating and fixing errors within the system this task made simpler when the programmer familiar with the various types of debugging tools available and how they can be used.

Tool type	Debugging tools	Descriptions	Examples of users and Drawbacks
Hardware	m-circuit emulator	Active device replaces microprocessor in system	* typically most expensive debug solutions, but has a lot of debugging capabilities * allows visibility and modifiability of internal memory registers variables etc real time * Similar to debuggers allows setting break point single stepping etc. * usually has a overlay memory to simulate Rom * processor dependent
	emulator	Active tools replaces Rom which cables connected to dual port RAM within Rom emulator simulates Rom. It is an Intermediate hardware device connected to the target via same cable (BDM) and connect:	* allows modification of contents in Rom * can set break points in Rom code and view code real time * processor dependent

Background debug mode (Bdm)	Bdm hardware on board and debugger on host connected with via serial cable to Bdm port commonly referred to as wiggler Bdm debugging sometimes referred to on-chip Debugging	⊛ Usually cheaper than ICE, but not as flexible as ICE ⊛ observe software execution unobtrusively in real time ⊛ can set breakpoints to stop software execution
IEEE 1149.1 Joint Test Action Group	JTAG compliant hardware on board	⊛ Similar to Bdm but not proprietary to specific architecture
IEEE-ISTO Nexus 5001	Option of JTAG port Nexus compliant port or both several layers of compliance	⊛ offers scalable debug functions depending on level of compliance of hardware

oscilloscope	passive analog device that graphs voltage versus time detecting exact voltage at a given time	* monitor up to two signals simultaneously * can set a trigger to capture voltage given specific conditions * processor independent
logic analyzer	passive device that capture and tracks multiple signals simultaneously and can graph them	* can be expensive * typically can only track two voltages (vcc and ground) * can store data.
voltmeter	measure voltage difference between two points on circuit	* to measure for particular voltage values * cheaper than other hardware tools.

	ohmmeter	measures resistance blw two points on circuit	⊛ cheaper than other hardware tools ⊛ to measure changes in current/voltage in terms of resistance
	multimeter	measures both voltage and resistance	⊛ same as volt and ohm meter
software	debugger	Functional debugging tool	⊛ depends on the debugger ⊛ loading/single stepping tracing code on target ⊛ can modify contents of RAM, typically cannot modify contents of Rom
	profiler	Collects the timing history selected variables registers	⊛ capture time-dependent behaviour of emulating

	monitor	debugging interface similar to ICE with debug software running on target and host part of monitor beside in Rom of target board	⊛ embedded oss can include monitor for particular architectures ⊛ similar to print statement but faster, less intrusive works better for soft real time deadlines but not hard real time.
	Instruction set simulator	Run on host and simulates master processor and memory	⊛ typically does not run at exact same speed of real target, but can estimate response and throughput times by taking into consideration difference blw host and target speeds ⊛. can simulate interrupt behavior.

System Boot-Up :

* System boot-up means that some type of Power ON or Reset source, such as an internal/external hard reset (generated by a check-stop error, the software watchdog, a loss of lock by the PLL, debugger, etc.) or an internal/external soft reset (generated by a debugger, application code, etc.), has occured.

* When power is applied to an embedded board (because of a reset), start-up code, also referred to as boot code, bootloader, bootstrap code, or BIOS (basic input/output system) depending on the architecture, in the system's ROM is loaded and executed by the master processor. Some embedded architectures have an internal program counter that is automatically configured with an address in ROM in which the start of the boot-up code is located, while others are hardware wired to start executing at a specific location in the memory.

* Boot code differs in length and functionality depending on where in the development cycle the board is, as well as the components of the actual platform that need initialization.
* The same general functions are performed by boot code across the various platforms, which are basically initializing the hardware, which includes disabling interrupts, initializing buses, setting the master and slave processors in a specific state, and initializing memory.
* After the hardware initialization sequence, executed via initialization device drivers, the remaining system software, if any, is then initialized. This additional code may exist in ROM, for a system that is being shipped out of the factory, or loaded from an external host platform.

5. Quality Assurance and Testing of the Design :-

* Quality assurance and testing is similar to debugging, except that the goals of debugging are to actually fix discovered bugs.
* Another main difference between debugging and testing the system is that debugging typically occurs when the developer encounters a problem in trying to complete a portion of the design, and then typically tests-to-pass the bug fix (meaning tests only to ensure the system minimally works under normal circumstances).
* With testing, on the other hand, bugs are discovered as a result of trying to break the system, including both testing-to-pass and testing-to-fail, where weakness in the system are probed.
* Testing can be further broken down to include, for example :
* Unit / Module testing :- incremental testing of individual elements within the system.

* Compatibility testing :- Testing that the element doesn't cause problems with other elements in the system.

	Black Box Testing	White Box Testing.
Static Testing	Testing the product specifications by : 1. Looking for high-level fundamental problems, oversights, omissions (pretending to be customer, research existing guidelines / standards, review and test similar software, etc.) 2. low-level specification testing by insuring completeness, accuracy, preciseness, consistency, relevance, feasibility, etc	Process of methodically reviewing hardware and code for bugs without executing it.
Dynamic Testing	Requires definition of what software and hardware does, includes :	Testing running system while looking at code, schematics, etc

* Data testing, which is checking info of user inputs and outputs
* Boundary condition testing, which is testing situations at edge of planned operational limits of the software
* internal boundary testing, which is testing powers-of-two, ASCII table
* Input testing, which is testing null, invalid data
* State testing, which is testing modes and transitions between modes software is in with state variables

e.g., race conditions, repetition testing (main reason is to discover memory leaks), stress (starving software = low memory, slow cpu, slow network), load (feed software = connect many peripherals, process large amount of data, web server has many clients accessing it, etc.).

Directly testing low-level and high-level based on detailed operational knowledge, accessing variables and memory dumps.

Looking for data reference errors, data declaration errors, computation errors, comparision errors, control flow errors, sub-routine parameter errors, I/O errors, etc.

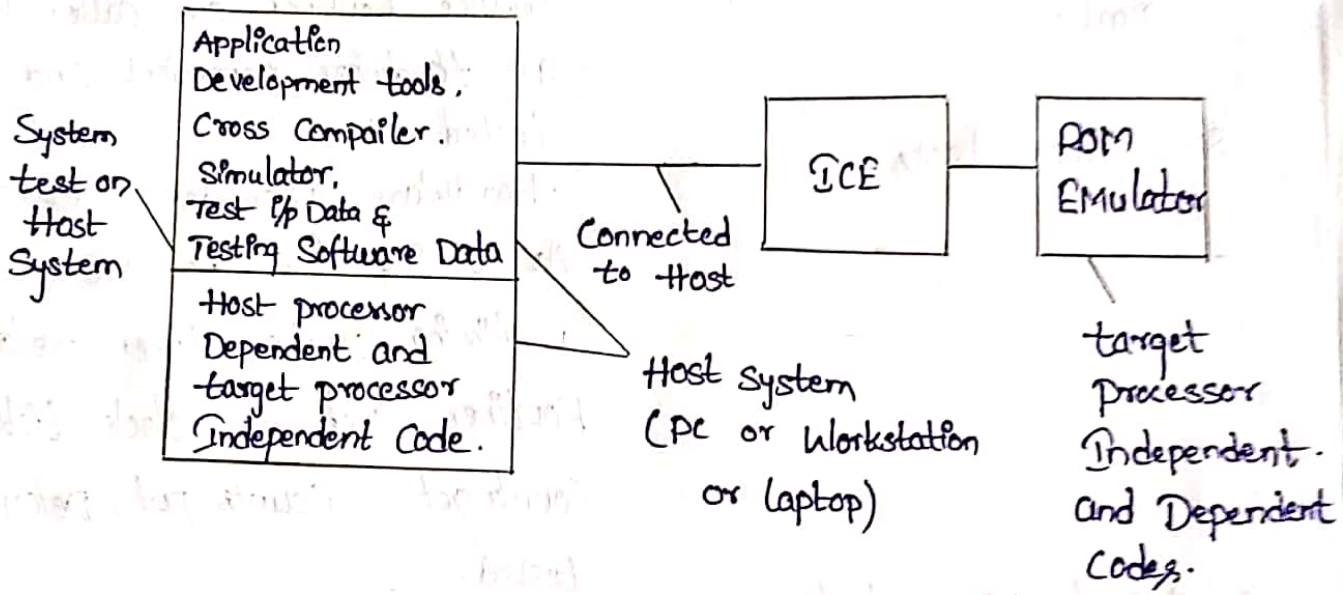
* Integration testing :- Incremental testing of integrated elements
* System testing :- Testing the entire embedded system with all elements integrated.
* Regression testing :- Rerunning previously passed tests after system modification.
* Manufacturing testing :- Testing to ensure that manufacturing of the system didn't introduce bugs.

Code inspection teams should have some combination of the following type of roles :

1. Moderator :- Responsible for managing the code inspection process and meeting(s).
2. Reader :- reads out-loud the source code and relative specifications for the operational investigation. should not be the developer that created the source code being inspected
3. Recorder :- Fills in code inspection check list report and documents any agreed upon open items.
4. Author :- helps explain source to code inspection team, to discuss errors found, and future rework that needs to be done. should be the developer of the source code being inspected.

(6) TESTING ON HOST MACHINE :-

We have two Systems With Diffrent CPU's or MicroController & Hardware Architecture. one System is Host And the other is the target. the Host is generally PC or Laptop or Workstation. target is Actually Hardware to be Used for Embedded System Under Development.



TESTING STEPS AT HOST MECHINE :-

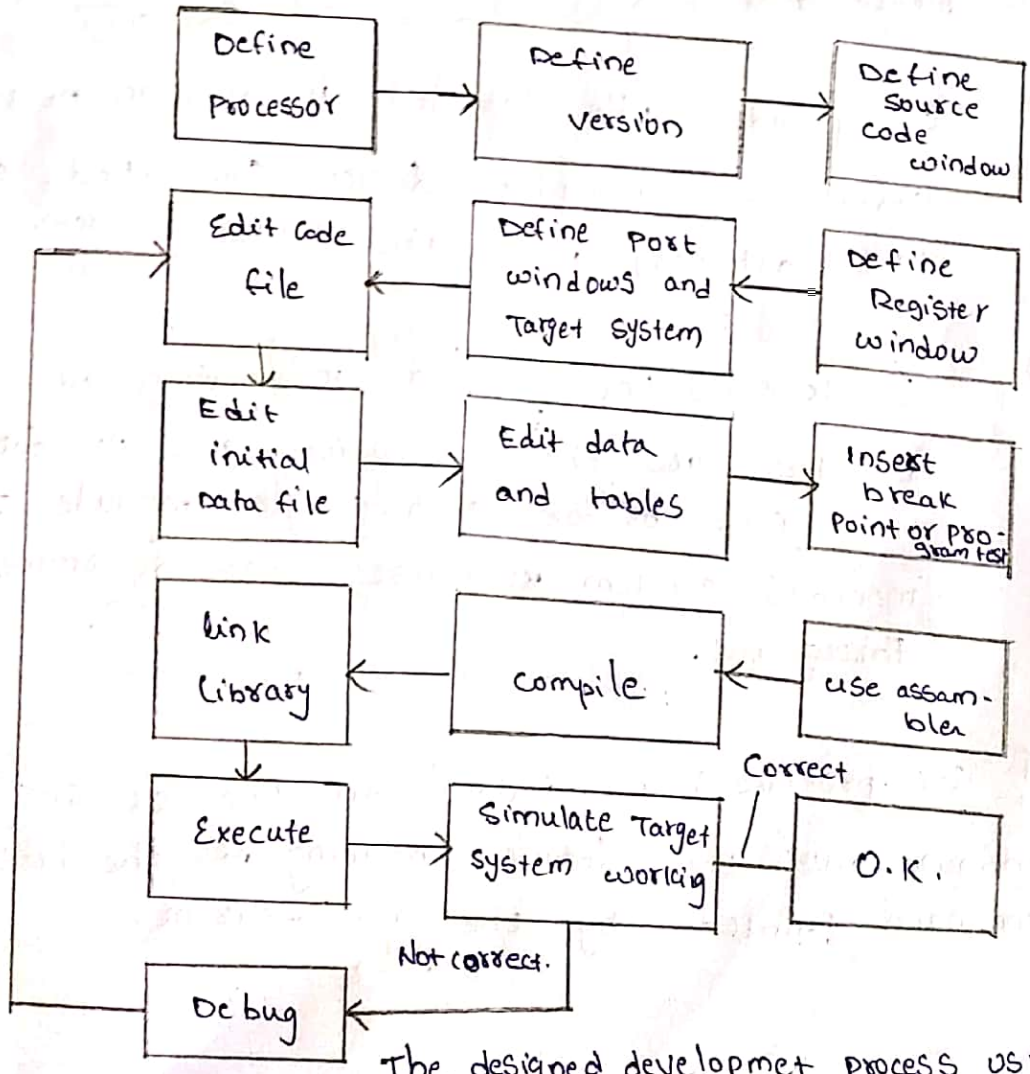
steps	Action.
1. Initial tests	Test Each Module or Segment at initial stage itself & on Host itself.
2. Test Data.	All possible Combinations of Data are designed and taken as test Data.

Steps	Action
3. Exception Condition tests.	Consider all possible exceptions for the test.
4. Test - 1.	Test the Hardware Independent Code.
5. Test - 2.	Test scaffold Software.
6. Test Interrupt Service Routines Hardware - Independent part.	Those sections of Interrupt Service Routines are called, which are Hardware Independent and tested.
7. Test Interrupt Service Routines Hardware - Dependent part.	Those sections of Interrupt Service Routines are called, which are Hardware Dependent and tested.
8. Timer tests.	Hardware Dependent Code As timing functions are used a timing Device. Timer related Routines such As clocks tick set, Counts get, Counts put, Delay are tested.
9. Assert Macro tests.	The use of an assert Macro is an important test technique. For Example, Consider a Command, 'assert (P pointer != NULL);' We insert the Codes in the program that check whether a Condition or a parameter actually turns true or false.

⑦ Simulators;

Simulators uses knowledge of target processor or microcontroller, and target system architecture on the host processor. simulator first does cross-compilation of the codes and places these into the host System RAM. The behaviour of the target system processor registers is also simulated the RAM.

It uses linker and locator to port the cross-compiled codes in RAM and functions like the code that would have run at the actual target system. Host system is a PC or work station or laptop and generally works in windows.



The designed developmet process using simulator.

Simulator features:-

a. It defines the processor or processing device family as well as its various versions for the target system.

b. It monitors the detailed information of a source code past with labels and symbolic arguments as the execution goes on for each single step.

c. It provides the detail information of the status of RAM and ports (simulated) of the defined target system as the Execution goes on for each single step.

d. It provides the detailed information of the status of peripheral devices (simulated, assumed to be attached) with the defined system.

e. It provides the detailed information of the registers are the execution goes on for each single step or for each single module. It also monitors system response and determines throughput.

f. It provides help window on the screen. A help window gives the detailed meaning of the present command pointed by the mice-cursor.

g:- It monitors the detailed information of the simulator commands as these are entered from the keyboard or selected from the menu.

h:- It facilitates synchronizing the internal periphorals and delays.

i:- It employs preempting RTOS scheduler support for the high priority tasks.

j:- It provides network driver and device driver support

8. * LABORATORY TOOLS :-

(1) Simple volt-ohm meter: Simple volt-ohm meter can be used to test the target hardware. It has two leads marked red and black. One end of each is connected to the meter and the other to points between which the voltage or resistance is to be measured.

(2) Simple LED Tests and logic probe:

→ Let us remember that a digital signal is simply a discre Signal between two voltage ranges. CMOS logic, '1' is th discrete range V_{DD} to 0.66 V_{DD} and '0' is 0.33V_{DD} V_{DD} where V_{DD} is usually 5 V with respect to V_{DD}

→ A logic probe is the ~~Smallest~~ Simplest hardware test device it is a handled pen like device with LEDs.

(3) Oscilloscope:-

Code downloaded hardware needs testing after completing the edit-test and debug cycle. using a Simulator or IDE An oscilloscope is a scope with a screen to display two signal voltages as a function of time it displays analog as well as digital signals as a function of time.

(4) Bit Rate Meter:-

A bit rate meter is a Measuring device that finds the number of '1's and '0's in the preselected time spans. How to measure the throughput the number of bytes per Second on a network? Assume that 0xA55A (binary 1010010101011010) is sent repeatedly as output bits.

(5) Logic Analyser:-

Logic analyser is a power tool to collect through multiple input lines (Say 24 or 48) from the buses, ports and records many bus transactions (about 128 or more) It displays these on the Monitor (Screen) to debug real-time triggering conditions. It helps in sequentially finding the Signals as the instructions execute with respect to a reference one of the bus Signal or clock Signal is taken as

the reference.

(6) In circuit emulator (ICE):-

ICE is a circuit for emulating the target system that remains independent of a particular targeted system processor usable during the development phase for most of the target systems that will incorporate a particular Microcontroller chip. it works independently as well as by connecting to the pc through a serial link. It is a target circuit minus target Microcontroller and microprocessor.

(7) Monitor:-

Monitor is a bebugging tools for actual target Microprocessor or Microcontroller in ICE ROM emulator or in target development board. It also lets host system debugging interface just like an ICE. Monitors from different source differ in their functioning.

— x —

— PRAWIN